

# 16-realtime-apps

## 16. Aplikasi Real-Time

**Level:** □ Intermediate

**Jam:** 8 (4 minggu × 2 sesi)

**Prasyarat:** REST API (express), JavaScript/TypeScript dasar

**Output:** Real-time chat app dengan WebSocket & Socket.IO + deployment siap scaling

---

### Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Paham perbedaan polling, SSE, dan WebSocket serta kapan pakai masing-masing
- Bikin WebSocket server & client pakai library ws
- Implementasi real-time chat dengan Socket.IO (emit, on, broadcast, rooms, namespace)
- Nambahin typing indicator, presence, dan notifikasi real-time
- Scaling WebSocket dengan Redis adapter
- Handle graceful shutdown, rate limiting, dan monitoring koneksi WebSocket

### Materi

| Sesi | Topik                                | File                                     |
|------|--------------------------------------|------------------------------------------|
| 1    | WebSocket Protocol & ws Library      | <a href="#">01-websocket-basics.md</a>   |
| 2    | Socket.IO — Events, Rooms, Namespace | <a href="#">02-socketio.md</a>           |
| 3    | Scaling & Production WebSocket       | <a href="#">03-scaling-production.md</a> |

---

## 1. Kenapa Perlu Real-Time?

HTTP request-response biasa sifatnya **client pull** — client minta, server kasih, koneksi putus. Kalau ada data baru di server, client nggak tau sampai dia request lagi.

Aplikasi real-time butuh **server push** — server kirim data ke client **tanpa diminta**. Contoh: chat, notifikasi, kolaborasi dokumen, live score.

Tiga teknik utama: **Polling**, **SSE** (Server-Sent Events), **WebSocket**.

## 2. Perbandingan: Polling vs SSE vs WebSocket

| Aspek                     | Polling (HTTP)               | SSE                         | WebSocket            |
|---------------------------|------------------------------|-----------------------------|----------------------|
| <b>Arah</b>               | Client → Server (request)    | Server → Client (1 arah)    | Bidirectional        |
| <b>Transport</b>          | HTTP biasa                   | HTTP + stream               | TCP (upgrade)        |
| <b>Latency</b>            | Tinggi (tergantung interval) | Rendah                      | Sangat rendah        |
| <b>Header per msg</b>     | Penuh (HTTP headers)         | Minimal                     | Minimal              |
| <b>Reconnect otomatis</b> | ✗                            | ☑ (built-in EventSource)    | ☑ (butuh library)    |
| <b>Dukungan browser</b>   | Semua browser                | Modern browser (kecuali IE) | Semua browser        |
| <b>Kompleksitas</b>       | Sederhana                    | Sedang                      | Agak kompleks        |
| <b>Kasus pakai</b>        | Fallback, legacy             | Notifikasi 1 arah, feed     | Chat, game real-time |

**Polling:** Client panggil endpoint tiap 1-5 detik. Boros bandwidth karena banyak request kosong.

**SSE:** Client buka satu koneksi HTTP, server kirim data kapan pun. Cocok untuk **notifikasi, feed berita, update harga saham**. Browser punya EventSource API native.

**WebSocket:** Koneksi TCP full-duplex. Client dan server bisa kirim data kapan saja. Standar untuk **chat, game real-time, kolaborasi**.

## 3. WebSocket Handshake (Penjelasan Sederhana)

WebSocket mulai sebagai permintaan HTTP biasa, lalu **di-upgrade** jadi koneksi WebSocket.



```

| Upgrade: websocket |
| Connection: Upgrade |
| Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ== |
| Sec-WebSocket-Version: 13 |
|
|<--- HTTP/1.1 101 Switching ---|
| Upgrade: websocket |
| Connection: Upgrade |
| Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o= |
|
|<===== koneksi WebSocket =====>|
| (full-duplex, frame biner) |

```

#### Langkah:

1. Client kirim HTTP GET dengan header Upgrade: websocket.
2. Header Sec-WebSocket-Key berisi base64 random (mencegah cache proxy).
3. Server terima, hitung Sec-WebSocket-Accept (SHA-1 key + GUID, lalu base64).
4. Server balas status **101 Switching Protocols**.
5. Koneksi HTTP **mati**, diganti protokol WebSocket di TCP yang sama.
6. Data dikirim dalam **frame** — ringan, tanpa HTTP headers.

Siswa nggak perlu ngoding handshake manual. **Socket.IO** (dan library WebSocket lain) urusin semua ini.

## 4. Socket.IO Setup + Express

Socket.IO = library WebSocket + fallback (long-polling kalau WebSocket diblokir). Ditambah fitur rooms, broadcast, namespace.

### 4.1 Instalasi

```

mkdir chat-app && cd chat-app
npm init -y
npm install express socket.io

```

### 4.2 Server Dasar

```

// server.ts
import express from 'express';
import http from 'http';
import { Server } from 'socket.io';

```

```

const app = express();
const server = http.createServer(app);
const io = new Server(server); // attach Socket.IO ke HTTP server

app.get('/', (req, res) => {
  res.sendFile(__dirname + '/index.html');
});

// Event ketika client terhubung
io.on('connection', (socket) => {
  console.log('User terhubung:', socket.id);

  socket.on('disconnect', () => {
    console.log('User putus:', socket.id);
  });
});

server.listen(3000, () => {
  console.log('Server jalan di http://localhost:3000');
});

```

### 4.3 Client Dasar (HTML + Socket.IO)

```

<!-- index.html -->
<!DOCTYPE html>
<html>
<head><title>Chat App</title></head>
<body>
  <script src="/socket.io/socket.io.js"></script>
  <script>
    const socket = io(); // otomatis connect ke server yang sama
    console.log('Socket ID:', socket.id);
  </script>
</body>
</html>

```

File socket.io.js disediakan otomatis oleh server Socket.IO di endpoint /socket.io/socket.io.js. Ngak perlu download terpisah.

---

## 5. Events: emit, on, broadcast, rooms

### 5.1 Emit & On (Kirim & Terima)

```
// Server
io.on('connection', (socket) => {
  // Menerima event dari client
  socket.on('pesan', (data) => {
    console.log('Dari client:', data);
  });

  // Kirim event ke client tertentu
  socket.emit('halo', 'Halo dari server!');
});

// Client
socket.emit('pesan', { teks: 'Halo server!' });
socket.on('halo', (msg) => {
  console.log(msg);
});
```

### 5.2 Broadcast (Kirim ke Semua Kecuali Pengirim)

```
// Server – kirim ke semua client kecuali pengirim
socket.broadcast.emit('user-menetik', { user: 'Budi' });

// Server – kirim ke SEMUA client (termasuk pengirim)
io.emit('pesan-baru', { teks: 'Announcement!' });
```

### 5.3 Rooms (Grup)

```
// Join room
socket.join('kelas-12-rpl');

// Kirim ke satu room aja
io.to('kelas-12-rpl').emit('tugas-baru', { judul: 'Buat REST API' });

// Keluar room
socket.leave('kelas-12-rpl');
```

#### Diagram alur:

```
io.emit()           → semua client
socket.emit()       → client itu aja
socket.broadcast.emit() → semua kecuali pengirim
io.to('room').emit() → semua di room itu
socket.broadcast.to('room').emit() → room lain kecuali pengirim
```

---

## 6. Build Simple Chat App (Step by Step)

### Step 1: Struktur Proyek

```
chat-app/  
├── server.ts  
├── package.json  
└── public/  
    └── index.html
```

### Step 2: Server.ts Lengkap

```
import express from 'express';  
import http from 'http';  
import { Server } from 'socket.io';  
import path from 'path';  
  
const app = express();  
const server = http.createServer(app);  
const io = new Server(server);  
  
app.use(express.static('public'));  
  
const chatHistory: { id: number; user: string; teks: string; waktu: string }[] = [];  
  
io.on('connection', (socket) => {  
  console.log('User join:', socket.id);  
  
  // Kirim history ke client baru  
  socket.emit('chat-history', chatHistory);  
  
  // Denger pesan baru  
  socket.on('chat-message', (data: { user: string; teks: string }) => {  
    const msg = {  
      id: Date.now(),  
      user: data.user,  
      teks: data.teks,  
      waktu: new Date().toLocaleTimeString()  
    };  
  
    chatHistory.push(msg);  
    if (chatHistory.length > 50) chatHistory.shift();  
  });  
});
```

```

    // Broadcast ke SEMUA client
    io.emit('chat-message', msg);
  });

  // Typing indicator
  socket.on('typing', (user: string) => {
    socket.broadcast.emit('user-typing', user);
  });

  socket.on('stop-typing', () => {
    socket.broadcast.emit('user-stop-typing');
  });

  socket.on('disconnect', () => {
    console.log('User leave:', socket.id);
    io.emit('user-offline', socket.id);
  });
});

server.listen(3000);

```

### Step 3: Client HTML

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Chat RPL</title>
  <style>
    body { font-family: Arial; max-width: 600px; margin: auto; padding: 20px; }
    #messages { border: 1px solid #ccc; height: 300px; overflow-y: auto; padding: 10px; }
    #typing { font-style: italic; color: #666; height: 20px; }
    input[type="text"] { width: 70%; padding: 8px; }
    button { padding: 8px 16px; }
  </style>
</head>
<body>
  <h2>Chat Room – RPL</h2>
  <div id="messages"></div>
  <div id="typing"></div>
  <input type="text" id="msgInput" placeholder="Ketik pesan..." autofocus />
  <button id="sendBtn">Kirim</button>

  <script src="/socket.io/socket.io.js"></script>
</script>

```

```

const socket = io();
const user = 'User-' + Math.random().toString(36).substr(2, 4);

function addMessage(msg) {
  const div = document.getElementById('messages');
  const el = document.createElement('div');
  el.innerHTML = `<strong>${msg.user}</strong> <small>${msg.waktu}</small><b>`
  div.appendChild(el);
  div.scrollTop = div.scrollHeight;
}

socket.on('chat-history', (history) => {
  history.forEach(addMessage);
});

socket.on('chat-message', addMessage);

function sendMessage() {
  const input = document.getElementById('msgInput');
  if (input.value.trim() === '') return;
  socket.emit('chat-message', { user, teks: input.value });
  input.value = '';
}

document.getElementById('sendBtn').onclick = sendMessage;
document.getElementById('msgInput').onkeypress = (e) => {
  if (e.key === 'Enter') sendMessage();
};

let typingTimer;
document.getElementById('msgInput').oninput = () => {
  socket.emit('typing', user);
  clearTimeout(typingTimer);
  typingTimer = setTimeout(() => socket.emit('stop-typing'), 1000);
};

socket.on('user-typing', (user) => {
  document.getElementById('typing').textContent = `${user} sedang mengetik..`
});

socket.on('user-stop-typing', () => {
  document.getElementById('typing').textContent = '';
});
</script>
</body>
</html>

```



#### Step 4: Jalankan

`npx tsx server.ts`

*# Buka `http://localhost:3000` di dua tab browser*

#### Cara kerja:

1. Client kirim chat-message berisi { user, teks }.
  2. Server terima, simpan ke history, broadcast `io.emit('chat-message', msg)`.
  3. Semua client (termasuk pengirim) dapet event dan render pesan.
  4. Input kosong setelah kirim, scroll otomatis ke bawah.
- 

## 7. Real-Time Notifications

Notifikasi berbeda dari chat — biasanya muncul sebagai **toast/popup** atau **badge counter**.

```
// Server – kirim notifikasi ke room tertentu
function kirimNotif(userId: string, judul: string, body: string) {
  io.to(`user-${userId}`).emit('notifikasi', {
    id: Date.now(),
    judul,
    body,
    icon: 'bell',
    waktu: new Date().toISOString()
  });
}

// Contoh: notifikasi tugas baru
kirimNotif('budil23', 'Tugas Baru', 'Buat presentasi RPL hari Jumat');

// Client – tangani notifikasi
socket.on('notifikasi', (notif) => {
  // Tampilkan sebagai toast
  tampilkanToast(notif.judul, notif.body);

  // Update badge counter
  notifCount++;
  document.getElementById('notifBadge').textContent = notifCount;

  // Optional: simpan di localStorage
  simpanNotif(notif);
});
```

## Tips Notifikasi

- **User-specific:** join room user-`\${userId}` setelah login agar notifikasi cuma ke orang tertentu.
- **Read/unread:** simpan status read di database, kirim unread count saat user connect.
- **Sound:** mainkan audio pendek (`new Audio('/notif.mp3').play()`) saat notifikasi masuk.

```
// Client – join room berdasarkan user setelah connect
socket.on('connect', () => {
  const userId = localStorage.getItem('userId');
  socket.emit('join-user-room', userId);
});

// Server
socket.on('join-user-room', (userId: string) => {
  socket.join(`user-${userId}`);
});
```

---

## 8. Live Collaboration Features

### 8.1 Typing Indicator

Sudah diimplementasi di chat app di atas. Prinsipnya:

Client: [input aktif] → emit('typing', username)  
→ setelah 1 detik tidak mengetik → emit('stop-typing')  
Server: broadcast('user-typing') kecuali pengirim  
Client: tampilkan teks "X sedang mengetik..."

### 8.2 Cursor Position (Live Cursor)

Berguna di dokumen kolaboratif (Google Docs-like). Kirim posisi kursor tiap kali user gerakin mouse/klik.

```
// Client – kirim posisi cursor
document.addEventListener('mousemove', (e) => {
  if (lastCursorSend && Date.now() - lastCursorSend < 50) return;
  lastCursorSend = Date.now();

  socket.emit('cursor-move', {
    x: e.clientX,
    y: e.clientY,
    user: username
  });
});
```

```

    });
  });

  // Client – render cursor orang lain
  const cursors: Record<string, HTMLElement> = {};

  socket.on('cursor-update', (data) => {
    if (!cursors[data.user]) {
      const el = document.createElement('div');
      el.className = 'remote-cursor';
      el.textContent = '□ ' + data.user;
      el.style.position = 'fixed';
      el.style.pointerEvents = 'none';
      el.style.zIndex = '9999';
      document.body.appendChild(el);
      cursors[data.user] = el;
    }

    cursors[data.user].style.left = data.x + 'px';
    cursors[data.user].style.top = data.y + 'px';
  });

  // Server
  socket.on('cursor-move', (data) => {
    socket.broadcast.emit('cursor-update', data);
  });

  // Hapus cursor saat user disconnect
  socket.on('disconnect', () => {
    io.emit('cursor-remove', socket.username);
  });

  .remote-cursor {
    font-size: 14px;
    background: rgba(255, 255, 255, 0.9);
    border: 1px solid #ccc;
    border-radius: 4px;
    padding: 2px 6px;
    transition: left 0.1s, top 0.1s;
  }

```

### 8.3 Presence (Online/Offline)

```

// Server – track user online
const onlineUsers = new Map<string, string>();

```

```

io.on('connection', (socket) => {
  socket.on('user-online', (username: string) => {
    onlineUsers.set(socket.id, username);
    io.emit('presence-update', Array.from(onlineUsers.values()));
  });

  socket.on('disconnect', () => {
    onlineUsers.delete(socket.id);
    io.emit('presence-update', Array.from(onlineUsers.values()));
  });
});

// Client
socket.emit('user-online', username);

socket.on('presence-update', (users) => {
  document.getElementById('online-count').textContent = users.length;
  document.getElementById('online-list').innerHTML = users.map(u => `<li> ${u}</li>`);
});

```

## 9. Kesimpulan

| Konsep                      | Intinya                                                                   |
|-----------------------------|---------------------------------------------------------------------------|
| <b>Polling vs SSE vs WS</b> | Polling sederhana tapi boros. SSE 1 arah ringan. WebSocket full-duplex    |
| <b>WebSocket handshake</b>  | HTTP di-upgrade ke TCP full-duplex lewat header Upgrade.                  |
| <b>Socket.IO</b>            | Library yang bikin WebSocket gampang + fallback otomatis.                 |
| <b>emit/on</b>              | Kirim & terima event real-time.                                           |
| <b>broadcast</b>            | Kirim ke semua client kecuali pengirim.                                   |
| <b>rooms</b>                | Grup channel untuk chat privat atau per-kelas.                            |
| <b>Chat app</b>             | Server denger chat-message, broadcast io.emit, client render.             |
| <b>Notifikasi</b>           | User-specific room, tampilkan toast + badge.                              |
| <b>Collaboration</b>        | Cursor position, typing indicator, presence — semua pakai event real-time |

## Output Akhir Modul

**Real-Time Chat App** — aplikasi chat real-time dengan WebSocket yang bisa discale secara horizontal pakai Redis adapter. Fitur: multiple rooms, typing indicator, presence, notifikasi, dan monitoring koneksi.

## AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Explain this WebSocket handshake step by step"
- "Debug this Socket.IO event — client emits but server doesn't receive"
- "Refactor this chat server to use TypeScript interfaces"
- "Generate load test script for this WebSocket server with 100 concurrent connections"
- "Review this Redis adapter config for production readiness"

*Modul ini ditulis untuk siswa SMK RPL yang sudah familiar dengan Express dan REST API. Selanjutnya: deployment WebSocket ke production (Redis adapter, sticky sessions, scaling).*

## 16.1 WebSocket Protocol & ws Library

## WebSocket Protocol

WebSocket adalah protokol komunikasi **full-duplex** di atas TCP. Berbeda dengan HTTP: sekali koneksi terbuka, client dan server bisa kirim data kapan aja tanpa overhead HTTP headers tiap request.

## Frame WebSocket

Data WebSocket dikirim dalam **frame** — unit data kecil dengan struktur:

[illegible]

```

|      Payload Data (variable length)      |
+-----+-----+-----+-----+-----+
+-----+

```

**Opcode penting:** | Opcode | Arti | |-----|-----| | 0x1 | Text frame (UTF-8) | | 0x2 | Binary frame | | 0x8 | Connection close | | 0x9 | Ping | | 0xA | Pong |

### Flow koneksi:

1. TCP handshake → HTTP upgrade → WebSocket terbuka
2. Kirim-terima frame (text/binary)
3. Ping/pong buat jaga koneksi tetap hidup (keepalive)
4. Close frame — salah satu pihak kirim frame close, diterima, koneksi ditutup

### Connection Lifecycle

| Client |                                  | Server      |
|--------|----------------------------------|-------------|
|        | --- HTTP Upgrade Request ----->  | - Handshake |
|        | <--- 101 Switching Protocols --- |             |
|        | === WebSocket connection ===>    | - Active    |
|        | <--- Ping (opcode 0x9) -----     |             |
|        | --- Pong (opcode 0xA) ----->     |             |
|        | --- Text message (opcode 0x1)>   |             |
|        | <--- Text message (opcode 0x1)-  |             |
|        | ---                              |             |
|        | --- Close frame (opcode 0x8) ->  | - Closing   |
|        | <--- Close frame (opcode 0x8) -  |             |
|        | TCP connection closed            | - Closed    |

---

### ws Library — WebSocket di Node.js

Library ws adalah implementasi WebSocket paling populer di Node.js. Ringan, cepat, tanpa overhead library tambahan.

### Instalasi

```

mkdir ws-chat && cd ws-chat
npm init -y
npm install ws typescript @types/ws tsx

```

## WebSocket Server Sederhana

```
// server.ts
import { WebSocketServer } from 'ws';

const wss = new WebSocketServer({ port: 8080 });

console.log('WebSocket server jalan di ws://localhost:8080');

wss.on('connection', (ws, req) => {
  console.log('Client terhubung dari:', req.socket.remoteAddress);

  // Kirim pesan ke client ini
  ws.send('Selamat datang di WebSocket server!');

  // Terima pesan dari client
  ws.on('message', (data) => {
    console.log('Terima:', data.toString());
    // Echo balik
    ws.send(`Server echo: ${data}`);
  });

  ws.on('close', () => {
    console.log('Client putus');
  });

  ws.on('error', (err) => {
    console.error('WebSocket error:', err.message);
  });
});
```

## WebSocket Client

```
// client.ts
import WebSocket from 'ws';

const ws = new WebSocket('ws://localhost:8080');

ws.on('open', () => {
  console.log('Terhubung ke server');
  ws.send('Halo dari client!');
});

ws.on('message', (data) => {
  console.log('Dari server:', data.toString());
});
```

```

ws.on('close', () => {
  console.log('Koneksi ditutup');
});

ws.on('error', (err) => {
  console.error('Error:', err.message);
});

// Kirim pesan tiap 3 detik
setInterval(() => {
  ws.send(`Pesan otomatis ${Date.now()}`);
}, 3000);

```

Jalankan:

```

# Terminal 1 – server
npx tsx server.ts

```

```

# Terminal 2 – client
npx tsx client.ts

```

## Broadcasting — Kirim ke Semua Client

```

// broadcast-server.ts
import { WebSocketServer, WebSocket } from 'ws';

const wss = new WebSocketServer({ port: 8080 });

wss.on('connection', (ws) => {
  console.log('Client baru. Total:', wss.clients.size);

  ws.on('message', (data) => {
    const message = data.toString();

    // Broadcast ke SEMUA client termasuk pengirim
    wss.clients.forEach((client) => {
      if (client.readyState === WebSocket.OPEN) {
        client.send(`Pesan: ${message}`);
      }
    });
  });
});

ws.on('close', () => {
  console.log('Client pergi. Total:', wss.clients.size);
});

```



```
});
```

### Broadcast Tanpa Pengirim

```
ws.on('message', (data) => {  
  const message = data.toString();  
  
  wss.clients.forEach((client) => {  
    // Kirim ke semua kecuali pengirim  
    if (client !== ws && client.readyState === WebSocket.OPEN) {  
      client.send(`User lain: ${message}`);  
    }  
  });  
});
```

### Ping/Pong Keepalive

```
import { WebSocketServer, WebSocket } from 'ws';  
  
const wss = new WebSocketServer({ port: 8080 });  
  
const interval = setInterval(() => {  
  wss.clients.forEach((ws) => {  
    // Kirim ping - client harus balas pong dalam 10 detik  
    if (ws.readyState === WebSocket.OPEN) {  
      ws.ping();  
    }  
  });  
}, 30000); // tiap 30 detik  
  
wss.on('connection', (ws) => {  
  let isAlive = true;  
  
  ws.on('pong', () => {  
    isAlive = true;  
  });  
  
  // Terminate client yang mati  
  const check = setInterval(() => {  
    if (!isAlive) {  
      console.log('Client mati, terminate');  
      ws.terminate();  
      clearInterval(check);  
      return;  
    }  
  })
```

```

        isAlive = false;
        ws.ping();
      }, 10000);
    });

wss.on('close', () => clearInterval(interval));

```

## WebSocket dengan HTTP Server Bersama

```

// hybrid-server.ts
import http from 'http';
import { WebSocketServer, WebSocket } from 'ws';

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('HTTP server jalan\n');
});

const wss = new WebSocketServer({ server });

wss.on('connection', (ws, req) => {
  console.log('WebSocket client terhubung');

  ws.on('message', (data) => {
    ws.send(`Echo: ${data}`);
  });
});

server.listen(3000, () => {
  console.log('Server jalan di http://localhost:3000 dan ws://localhost:3000');
});

```

---

## Latihan

1. **Chat server with ws:** Bikin WebSocket server yang bisa handle multiple client. Server broadcast semua pesan ke semua client. Setiap client punya username unik. Format: { user: string, message: string, time: string }.
2. **Private message:** Tambah fitur DM — client kirim pesan dengan target ID { to: "socket-id", message: "halo" }. Server forward cuma ke socket target.
3. **WebSocket ping monitor:** Bikin server yang track round-trip time (RTT) tiap client. Catat waktu kirim ping, hitung selisih pas

pong balik. Log client yang RTT-nya > 500ms.

4. **Room system manual:** Implementasi room system tanpa library — client kirim { command: "join", room: "lobby" }. Server track client per room. Broadcast dalam room aja. Client bisa join multiple rooms.

## 16.2 Socket.IO — Events, Rooms, Namespace

Socket.IO adalah library real-time yang dibangun di atas WebSocket dengan banyak fitur tambahan: fallback transport (long-polling), auto-reconnect, rooms, namespace, dan middleware.

### Setup Express + Socket.IO

#### Instalasi

```
mkdir socketio-chat && cd socketio-chat
npm init -y
npm install express socket.io cors
npm install -D typescript @types/express @types/cors @types/node tsx
```

#### Server Dasar dengan CORS

```
// server.ts
import express from 'express';
import http from 'http';
import { Server } from 'socket.io';

const app = express();
const server = http.createServer(app);

const io = new Server(server, {
  cors: {
    origin: ['http://localhost:5173', 'http://localhost:3000'],
    methods: ['GET', 'POST']
  }
});

app.get('/', (req, res) => {
  res.json({ status: 'Socket.IO server running' });
});
```

```

io.on('connection', (socket) => {
  console.log('User connected:', socket.id);

  socket.on('disconnect', (reason) => {
    console.log('User disconnected:', socket.id, 'reason:', reason);
  });
});

server.listen(3000, () => {
  console.log('Server jalan di http://localhost:3000');
});

```

### Client Setup (HTML Vanilla)

```

<!DOCTYPE html>
<html>
<head>
  <title>Socket.IO Client</title>
  <script src="https://cdn.socket.io/4.7.5/socket.io.min.js"></script>
</head>
<body>
  <script>
    const socket = io('http://localhost:3000', {
      transports: ['websocket'], // force WebSocket aja, skip long-polling
      autoConnect: true         // default: langsung connect
    });

    socket.on('connect', () => {
      console.log('Connected:', socket.id);
    });

    socket.on('disconnect', (reason) => {
      console.log('Disconnected:', reason);
    });
  </script>
</body>
</html>

```

### Client Setup (TypeScript/React)

```

// client.ts
import { io, Socket } from 'socket.io-client';

const socket: Socket = io('http://localhost:3000', {
  transports: ['websocket'],

```

```
    reconnection: true,  
    reconnectionAttempts: 5,  
    reconnectionDelay: 1000  
  });  
  
export default socket;
```

---

## Events — Emit & On

### Event Dasar

```
// Server  
io.on('connection', (socket) => {  
  // One-way – kirim event tanpa butuh ack  
  socket.emit('welcome', 'Halo dari server!');  
  
  // With acknowledgment – client harus ack  
  socket.emit('question', 'Siapa nama kamu?', (answer: string) => {  
    console.log('User menjawab:', answer);  
  });  
  
  // Listen event  
  socket.on('message', (data: { user: string; text: string }) => {  
    console.log(`${data.user}: ${data.text}`);  
  });  
});  
  
// Client  
socket.on('welcome', (msg) => console.log(msg));  
  
socket.emit('question', 'Apa kabar?', (response) => {  
  console.log('Server reply:', response);  
});
```

### Data Types dengan Interface

```
// types.ts  
export interface ChatMessage {  
  id: string;  
  userId: string;  
  username: string;  
  text: string;  
  timestamp: number;  
}
```

```

export interface TypingEvent {
  userId: string;
  username: string;
  roomId: string;
}

// Server – emit dengan type
io.emit('new-message', {
  id: crypto.randomUUID(),
  userId: 'abc123',
  username: 'Budi',
  text: 'Halo semua!',
  timestamp: Date.now()
}) satisfies ChatMessage);

```

---

## Rooms

Room adalah channel internal Socket.IO. Client bisa join/leave room, dan server bisa kirim pesan ke room tertentu. **Tidak perlu join room untuk kirim pesan ke room.**

### Join & Leave Room

```

io.on('connection', (socket) => {
  // Join room
  socket.on('join-room', (roomId: string, username: string) => {
    socket.join(roomId);
    console.log(`${username} (${socket.id}) join room ${roomId}`);

    // Beri tau room lain
    socket.to(roomId).emit('user-joined', { userId: socket.id, username });
  });

  // Leave room
  socket.on('leave-room', (roomId: string) => {
    socket.leave(roomId);
    socket.to(roomId).emit('user-left', { userId: socket.id });
  });
});

```

### Kirim ke Room

```
// Ke semua dalam room (termasuk pengirim)
io.to('game-room-1').emit('game-update', { score: 100 });

// Ke semua dalam room kecuali pengirim
socket.to('game-room-1').emit('player-move', { x: 10, y: 20 });

// Ke multiple rooms
io.to('room-1').to('room-2').emit('broadcast', 'Pesan ke dua room');

// Ke semua room kecuali room tertentu
io.except('room-admin').emit('public-announcement', { text: 'Maintenance nanti m
```

### List Clients di Room

```
io.on('connection', async (socket) => {
  // Dapatkan semua socket di room tertentu
  const sockets = await io.in('game-room-1').fetchSockets();
  console.log('Client di game-room-1:', sockets.length);

  // Dapatkan semua room yang active
  const rooms = io.sockets.adapter.rooms;
  console.log('Active rooms:', rooms);
});
```

---

## Namespace

Namespace adalah channel terpisah di Socket.IO. Bedanya namespace dengan room: namespace punya **connection sendiri**, event listener terisolasi, berbeda URL path.

### Kenapa Pakai Namespace?

- /chat — untuk fitur chat
- /admin — untuk monitoring/admin
- /game — untuk game state
- Setiap namespace punya middleware sendiri, room sendiri, koneksi sendiri

### Setup Namespace

```
const server = http.createServer(app);
const io = new Server(server);
```

```

// Default namespace '/'
io.on('connection', (socket) => {
  console.log('Default namespace:', socket.id);
});

// Chat namespace
const chatNs = io.of('/chat');
chatNs.on('connection', (socket) => {
  console.log('Chat namespace:', socket.id);

  socket.on('send-message', (msg) => {
    chatNs.emit('new-message', msg);
  });
});

// Admin namespace
const adminNs = io.of('/admin');
adminNs.on('connection', (socket) => {
  console.log('Admin connected:', socket.id);
  adminNs.emit('stats', { activeUsers: 42 });
});

```

## Client Connect ke Namespace

```

// Client – connect ke namespace tertentu
const chatSocket = io('http://localhost:3000/chat');
const adminSocket = io('http://localhost:3000/admin');

chatSocket.emit('send-message', { text: 'Halo' });
adminSocket.on('stats', (data) => console.log(data));

```

---

## Middleware — Authentication

### Auth Middleware per Connection

```

import { Socket } from 'socket.io';

// Middleware – jalan SETIAP ada koneksi baru
io.use((socket: Socket, next) => {
  const token = socket.handshake.auth?.token;

  if (!token) {
    return next(new Error('Authentication required'));
  }
});

```



```

    }

    try {
      // Verify token (contoh: JWT sederhana)
      const decoded = verifyToken(token);
      socket.data.user = decoded; // simpan di socket.data
      next();
    } catch (err) {
      next(new Error('Invalid token'));
    }
  });

  io.on('connection', (socket) => {
    console.log('Authenticated user:', socket.data.user?.username);
    socket.emit('welcome', `Selamat datang ${socket.data.user?.username}!`);
  });

```

### Client Kirim Auth Token

```

// Client – kirim token saat connect
const socket = io('http://localhost:3000', {
  auth: {
    token: 'jwt-token-123'
  }
});

socket.on('connect_error', (err) => {
  console.error('Gagal connect:', err.message);
});

```

### Per-Namespace Middleware

```

const adminNs = io.of('/admin');

adminNs.use((socket, next) => {
  const role = socket.handshake.auth?.role;
  if (role !== 'admin') {
    return next(new Error('Only admins allowed'));
  }
  next();
});

adminNs.on('connection', (socket) => {
  socket.emit('admin-panel', { secret: 'classified' });
});

```

---

## Full Chat App — Socket.IO Version

### Struktur

```
socketio-chat/  
├── server.ts  
├── client/  
│   └── index.html  
└── package.json
```

### server.ts

```
import express from 'express';  
import http from 'http';  
import { Server, Socket } from 'socket.io';  
import cors from 'cors';  
  
const app = express();  
app.use(cors());  
app.use(express.static('client'));  
  
const server = http.createServer(app);  
const io = new Server(server, {  
  cors: { origin: '*' }  
});  
  
interface Message {  
  id: string;  
  username: string;  
  text: string;  
  timestamp: number;  
  roomId: string;  
}  
  
const messages = new Map<string, Message[]>(); // roomId → messages  
  
// Auth middleware  
io.use((socket, next) => {  
  const username = socket.handshake.auth?.username;  
  if (!username || username.length < 2) {  
    return next(new Error('Username minimal 2 karakter'));  
  }  
  socket.data.username = username;  
});
```

```

    next();
  });

io.on('connection', (socket: Socket) => {
  const username = socket.data.username;
  console.log(`${username} connected (${socket.id})`);

  socket.on('join-room', (roomId: string) => {
    // Leave previous rooms
    for (const room of socket.rooms) {
      if (room !== socket.id) socket.leave(room);
    }

    socket.join(roomId);
    socket.data.roomId = roomId;

    // Kirim history room
    const roomMessages = messages.get(roomId) || [];
    socket.emit('room-history', roomMessages);

    // Notify room
    socket.to(roomId).emit('user-joined', { username, userId: socket.id });
    io.to(roomId).emit('room-users', getUsersInRoom(io, roomId));
  });

  socket.on('send-message', (text: string) => {
    const roomId = socket.data.roomId;
    if (!roomId || !text.trim()) return;

    const msg: Message = {
      id: crypto.randomUUID(),
      username,
      text: text.trim(),
      timestamp: Date.now(),
      roomId
    };

    // Simpan history (max 100 per room)
    if (!messages.has(roomId)) messages.set(roomId, []);
    const roomMsgs = messages.get(roomId)!;
    roomMsgs.push(msg);
    if (roomMsgs.length > 100) roomMsgs.shift();

    io.to(roomId).emit('new-message', msg);
  });
});

```



```

#main { flex: 1; display: flex; flex-direction: column; }
#messages { flex: 1; overflow-y: auto; padding: 20px; }
#input-area { display: flex; padding: 10px; border-top: 1px solid #ddd; }
#msgInput { flex: 1; padding: 10px; font-size: 16px; }
#sendBtn { padding: 10px 20px; background: #0066ff; color: white; border: none; }
#rooms { list-style: none; }
#rooms li { padding: 8px; cursor: pointer; border-radius: 4px; margin: 2px 0; }
#rooms li:hover { background: #ddd; }
#rooms li.active { background: #0066ff; color: white; }
.msg { margin: 8px 0; padding: 8px; border-radius: 8px; background: #f0f0f0; }
.msg.own { background: #0066ff; color: white; margin-left: auto; }
.msg.user { font-size: 12px; font-weight: bold; }
.msg.time { font-size: 10px; color: #666; }
.msg.own.time { color: #ccc; }
#typing { height: 24px; padding: 0 20px; font-style: italic; color: #666; }
</style>
</head>
<body>
  <div id="sidebar">
    <h3>Rooms</h3>
    <ul id="rooms">
      <li data-room="umum" class="active"># umum</li>
      <li data-room="rpl"># rpl</li>
      <li data-room="ngoding"># ngoding</li>
    </ul>
    <h4>Online (<span id="online-count">0</span>)</h4>
    <ul id="online-users"></ul>
  </div>

  <div id="main">
    <div id="messages"></div>
    <div id="typing"></div>
    <div id="input-area">
      <input type="text" id="msgInput" placeholder="Ketik pesan..." />
      <button id="sendBtn">Kirim</button>
    </div>
  </div>

  <script>
    const username = prompt('Masukkan username:') || 'User-' + Math.random().toString(36).substr(2, 10);
    const socket = io('http://localhost:3000', {
      auth: { username }
    });

    let currentRoom = 'umum';

```

```

socket.on('connect_error', (err) => {
  alert('Gagal connect: ' + err.message);
});

socket.on('connect', () => {
  socket.emit('join-room', currentRoom);
});

// Room switching
document.querySelectorAll('#rooms li').forEach(el => {
  el.addEventListener('click', () => {
    document.querySelector('#rooms li.active')?.classList.remove('active');
    el.classList.add('active');
    currentRoom = el.dataset.room;
    document.getElementById('messages').innerHTML = '';
    socket.emit('join-room', currentRoom);
  });
});

// Messages
socket.on('room-history', (msgs) => {
  const container = document.getElementById('messages');
  container.innerHTML = '';
  msgs.forEach(addMessage);
});

socket.on('new-message', addMessage);

function addMessage(msg) {
  const div = document.createElement('div');
  div.className = 'msg' + (msg.username === username ? ' own' : '');
  div.innerHTML = `
    <div class="user">${msg.username}</div>
    <div>${msg.text}</div>
    <div class="time">${new Date(msg.timestamp).toLocaleTimeString()}</div>
  `;
  document.getElementById('messages').appendChild(div);
  div.scrollIntoView({ behavior: 'smooth' });
}

// Send
function sendMessage() {
  const input = document.getElementById('msgInput');
  if (input.value.trim()) {
    socket.emit('send-message', input.value);
    input.value = '';
  }
}

```

```

        socket.emit('stop-typing');
    }
}
document.getElementById('sendBtn').onclick = sendMessage;
document.getElementById('msgInput').onkeypress = (e) => {
    if (e.key === 'Enter') sendMessage();
};

// Typing
let typingTimer;
document.getElementById('msgInput').oninput = () => {
    socket.emit('typing');
    clearTimeout(typingTimer);
    typingTimer = setTimeout(() => socket.emit('stop-typing'), 1000);
};

socket.on('user-typing', (user) => {
    document.getElementById('typing').textContent = `${user} sedang mengetik..`
});
socket.on('user-stop-typing', (user) => {
    document.getElementById('typing').textContent = '';
});

// Online users
socket.on('room-users', (users) => {
    document.getElementById('online-count').textContent = users.length;
    document.getElementById('online-users').innerHTML = users.map(u => `<li>
});

socket.on('user-joined', ({ username }) => {
    console.log(`${username} join room`);
});

socket.on('user-left', ({ username }) => {
    console.log(`${username} left room`);
});
</script>
</body>
</html>

```

---

## Latihan

1. **Typing indicator per room:** Modifikasi typing indicator supaya broadcast cuma di dalam room yang sama (bukan semua names-

pace). Pakai `socket.to(roomId).emit()`.

2. **Admin monitoring dashboard:** Bikin namespace `/admin` yang nampilin: total connected clients, jumlah client per room, pesan per detik. Pakai `io.engine.clientsCount` dan event counter.
3. **Direct message (DM):** Implementasi DM di atas Socket.IO. Client kirim `{ to: "socketId", text: "halo" }`. Server forward ke target. Hint: pakai `io.to(socketId).emit()`.
4. **File sharing via Base64:** Tambah fitur kirim gambar di chat. Client baca file `→ FileReader.readAsDataURL() → emit send-image → server broadcast ke room sebagai `.

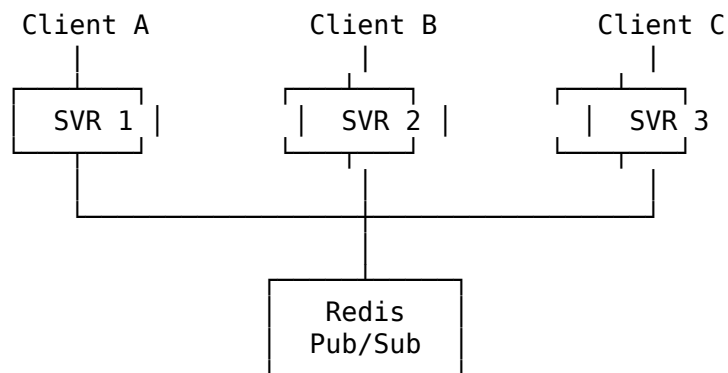
## 16.3 Scaling & Production WebSocket

WebSocket itu **stateful** — server nyimpen koneksi client di memory. Kalau scale ke multiple server, client yang connect ke server A **nggak bisa** dapet pesan dari server B. Solusi: **Redis adapter**.

### Redis Adapter untuk Socket.IO

Redis Adapter bikin semua instance Socket.IO share state via Redis Pub/Sub. Pesan dari server mana pun diteruskan ke semua server lain.

#### Arsitektur



- Socket.IO server A broadcast `→` publish ke Redis channel
- Redis kirim ke semua subscriber (termasuk server B, C)
- Server B & C terima, emit ke client mereka masing-masing



## Instalasi

```
npm install @socket.io/redis-adapter ioredis
npm install -D @types/ioredis
```

## Setup Redis Adapter

```
// server.ts
import express from 'express';
import http from 'http';
import { Server } from 'socket.io';
import { createAdapter } from '@socket.io/redis-adapter';
import { Redis } from 'ioredis';

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: '*' }
});

// Koneksi Redis
const pubClient = new Redis({
  host: process.env.REDIS_HOST || 'localhost',
  port: Number(process.env.REDIS_PORT) || 6379,
  password: process.env.REDIS_PASSWORD
});

const subClient = pubClient.duplicate(); // koneksi terpisah untuk subscribe

// Pasang Redis adapter
io.adapter(createAdapter(pubClient, subClient));

io.on('connection', (socket) => {
  console.log('User connected:', socket.id, 'on server:', process.pid);

  socket.on('send-message', (msg) => {
    // Broadcast ke semua server via Redis
    io.emit('new-message', msg);
  });
});

server.listen(3000, () => {
  console.log(`Server ${process.pid} jalan di http://localhost:3000`);
});
```

## Verifikasi Redis Adapter

```
# Terminal 1 – server port 3001
REDIS_HOST=localhost PORT=3001 node server.js

# Terminal 2 – server port 3002
REDIS_HOST=localhost PORT=3002 node server.js

# Kedua server share state via Redis
# Client connect ke server mana pun, dapet pesan dari semua
```

## Redis Adapter Options

```
io.adapter(createAdapter(pubClient, subClient, {
  // Key prefix di Redis (biar gak bentrok dengan app lain)
  key: 'chat-app:socket.io',

  // Requests key-value store di Redis (untuk fetchSockets dll)
  requestsTimeout: 5000 // default 5000ms
}));
```

---

## Sticky Sessions vs Redis Adapter

### Masalahnya

WebSocket upgrade request harus ke **server yang sama** yang handle HTTP request pertama. Kalau load balancer kirim ke server berbeda, handshake gagal.

### Solusi 1: Sticky Sessions (Cookie-based)

Load balancer kirim semua request dari satu client ke server yang sama berdasarkan cookie.

#### Nginx config:

```
upstream socket_backend {
    ip_hash; # Hash based on IP – sticky session
    server 127.0.0.1:3001;
    server 127.0.0.1:3002;
    server 127.0.0.1:3003;
}

server {
    listen 80;
```

```

server_name chat.example.com;

location / {
    proxy_pass http://socket_backend;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_read_timeout 86400; # 24 jam – koneksi WebSocket
}

```

## Solusi 2: Redis Adapter (Recommended)

Redis adapter ngasih **horizontal scaling tanpa sticky sessions**.  
Load balancer pake round-robin aja.

```

upstream socket_backend {
    # Round-robin – tanpa sticky session
    server 127.0.0.1:3001;
    server 127.0.0.1:3002;
    server 127.0.0.1:3003;
}

server {
    listen 80;
    server_name chat.example.com;

    location /socket.io/ {
        proxy_pass http://socket_backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_read_timeout 86400;
    }
}

```

## Perbandingan

| Aspek                | Sticky Sessions                   | Redis Adapter    |
|----------------------|-----------------------------------|------------------|
| <b>Setup</b>         | Konfigurasi LB aja                | Butuh Redis serv |
| <b>State sharing</b> | □ Tiap server punya state sendiri | □ State terpusat |

| Aspek                   | Sticky Sessions                                          | Redis Adapter     |
|-------------------------|----------------------------------------------------------|-------------------|
| <b>Failure handling</b> | Kalau server mati, client di server itu disconnect semua | Redis handle fail |
| <b>Scaling</b>          | Bisa scale, tapi client terpaku ke server tertentu       | Scale horizontal  |
| <b>Resource</b>         | 0 tambahan                                               | Butuh memory R    |
| <b>Recommended</b>      | Prototype / dev                                          | <b>Production</b> |

## Monitoring Koneksi WebSocket

### Metrics dengan socket.io-prometheus

```
npm install @socket.io/prometheus
```

```
import { Server } from 'socket.io';
import { Instrumentation } from '@socket.io/prometheus';

const io = new Server(server);
const instrumentation = new Instrumentation(io, {
  metrics: {
    connectedClients: true,
    events: true,
    rooms: true
  }
});

// Endpoint metrics – dipanggil Prometheus
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', instrumentation.contentType);
  res.end(await instrumentation.metrics());
});
```

### Manual Metrics

```
import { Server } from 'socket.io';

class SocketMonitor {
  private stats = {
    totalConnections: 0,
    activeConnections: 0,
    messagesSent: 0,
    messagesReceived: 0,
    rooms: 0
  };
};
```

```

constructor(private io: Server) {
  io.engine.on('connection', () => {
    this.stats.totalConnections++;
    this.stats.activeConnections++;
  });

  io.on('connection', (socket) => {
    socket.on('disconnect', () => {
      this.stats.activeConnections--;
    });

    // Track messages per socket
    socket.on('*', (event: string, ...args: any[]) => {
      this.stats.messagesReceived++;
    });
  });
}

getStats() {
  return {
    ...this.stats,
    currentClients: this.io.engine.clientsCount,
    rooms: this.io.sockets.adapter.rooms.size,
    uptime: process.uptime()
  };
}

// Expose via HTTP
app.get('/stats', (req, res) => {
  res.json(monitor.getStats());
});

```

### Logging Event

```

io.on('connection', (socket) => {
  // Log semua event yang masuk (debug mode)
  socket.use([event, ...args], next) => {
    console.log(`[${new Date().toISOString()}] ${socket.id}: ${event}`, args);
    next();
  });
});

```

---

## Graceful Shutdown

Pas server mati (deploy baru, restart), WebSocket client harus dikasih tau biar reconnect ke server lain.

```
import { Server } from 'socket.io';

const io = new Server(server);

function setupGracefulShutdown() {
  const shutdown = async (signal: string) => {
    console.log(`\n${signal} received - shutting down gracefully...`);

    // 1. Stop terima koneksi baru
    server.close();

    // 2. Kirim notifikasi ke semua client
    io.emit('server-shutdown', {
      message: 'Server akan restart dalam 5 detik',
      reconnectAfter: 5000
    });

    // 3. Disconnect semua socket dengan delay
    setTimeout(async () => {
      const sockets = await io.fetchSockets();
      for (const socket of sockets) {
        socket.disconnect(true);
      }

      // 4. Close Redis adapter
      // pubClient.quit();
      // subClient.quit();

      process.exit(0);
    }, 5000); // kasih waktu client reconnect
  };

  process.on('SIGTERM', () => shutdown('SIGTERM'));
  process.on('SIGINT', () => shutdown('SIGINT'));
}

setupGracefulShutdown();
```

## Client Handle Shutdown

```
// Client
socket.on('server-shutdown', (data) => {
  console.log('Server akan restart:', data.message);

  // Tampilkan pesan ke user
  showToast('Server maintenance, reconnect otomatis...');

  // Socket.IO auto-reconnect akan connect ke server baru
  // setelah delay sesuai data.reconnectAfter
  socket.io.opts.reconnectionDelay = data.reconnectAfter;
});
```

---

## Rate Limiting WebSocket

WebSocket bisa di-spam — client kirim ribuan pesan per detik. Harus di-rate limit.

### Per-Socket Rate Limiter

```
import { Socket } from 'socket.io';

class RateLimiter {
  private clients = new Map<string, { count: number; resetAt: number }>();

  constructor(
    private maxRequests: number = 50,    // max pesan per window
    private windowMs: number = 10000    // window 10 detik
  ) {}

  check(socketId: string): boolean {
    const now = Date.now();
    const entry = this.clients.get(socketId);

    if (!entry || now > entry.resetAt) {
      this.clients.set(socketId, { count: 1, resetAt: now + this.windowMs });
      return true;
    }

    if (entry.count >= this.maxRequests) {
      return false; // rate limited
    }
  }
}
```

```

    entry.count++;
    return true;
  }

  cleanup() {
    const now = Date.now();
    for (const [id, entry] of this.clients) {
      if (now > entry.resetAt) this.clients.delete(id);
    }
  }
}

// Middleware rate limit per event
const limiter = new RateLimiter(30, 10000); // 30 pesan per 10 detik
setInterval(() => limiter.cleanup(), 60000);

io.on('connection', (socket) => {
  socket.use(([event, ...args], next) => {
    if (event === 'send-message' || event === 'typing') {
      if (!limiter.check(socket.id)) {
        socket.emit('rate-limited', 'Terlalu cepat! Tunggu sebentar.');
```

```

        return; // BLOCK event
      }
    }
    next();
  });
});
```

## Token Bucket Algorithm

```

class TokenBucket {
  private tokens: number;
  private lastRefill: number;

  constructor(
    private maxTokens: number, // bucket capacity
    private refillRate: number, // token per detik
    private refillInterval: number = 1000
  ) {
    this.tokens = maxTokens;
    this.lastRefill = Date.now();
  }

  consume(count: number = 1): boolean {
    this.refill();
```



```

    if (this.tokens >= count) {
        this.tokens -= count;
        return true;
    }
    return false;
}

private refill() {
    const now = Date.now();
    const elapsed = now - this.lastRefill;
    const tokensToAdd = (elapsed / this.refillInterval) * this.refillRate;
    this.tokens = Math.min(this.maxTokens, this.tokens + tokensToAdd);
    this.lastRefill = now;
}
}

// Pakai di middleware
const buckets = new Map<string, TokenBucket>();

io.on('connection', (socket) => {
    buckets.set(socket.id, new TokenBucket(20, 5)); // 20 burst, 5/sec refill

    socket.on('send-message', (data) => {
        const bucket = buckets.get(socket.id);
        if (!bucket?.consume()) {
            return socket.emit('rate-limited', 'Slow down!');
        }
        // Proses pesan
        io.emit('new-message', data);
    });

    socket.on('disconnect', () => buckets.delete(socket.id));
});

```

---

## Production Checklist

- ☐ **Redis adapter** terpasang dan terverifikasi (kirim pesan dari server A, terima di server B)
- ☐ **Graceful shutdown** — client bisa reconnect tanpa kehilangan data
- ☐ **Rate limiting** — cegah spam per socket
- ☐ **Monitoring** — metrics (connected clients, events/sec, error rate)
- ☐ **Logging** — log connection/disconnection dengan timestamp +

- reason
- ❑ **CORS** — batasi origin yang boleh connect
- ❑ **Auth middleware** — validasi token tiap koneksi
- ❑ **Timeout** — pingTimeout dan pingInterval di Socket.IO
- ❑ **Max clients** — batasi max koneksi per server
- ❑ **HTTPS/WSS** — WebSocket harus pakai TLS di production

### Recommended Socket.IO Config

```
const io = new Server(server, {
  // CORS
  cors: {
    origin: process.env.CORS_ORIGIN?.split(',') || ['http://localhost:5173'],
    credentials: true
  },

  // Ping/pong
  pingInterval: 25000,      // Kirim ping tiap 25 detik
  pingTimeout: 20000,      // Anggap mati kalau 20 detik gak ada pong

  // Transport
  transports: ['websocket', 'polling'], // polling sebagai fallback

  // Max
  maxHttpBufferSize: 1e6, // 1 MB max message size

  // Allow request
  allowRequest: (req, callback) => {
    // Filter request berdasarkan origin atau header
    const origin = req.headers.origin;
    if (origin && !origin.includes('example.com')) {
      return callback(null, false); // reject
    }
    callback(null, true);
  }
});
```

---

### Latihan

1. **Multi-server chat dengan Redis:** Bikin 2 instance server Socket.IO (port 3001, 3002) dengan Redis adapter. Client connect ke port random. Kirim pesan dari client A, verifikasi client B terima meskipun beda server.

2. **Rate limiter dashboard:** Bikin monitoring yang nampilin: jumlah client per server, total pesan per detik, client yang kena rate limit. Pakai Redis buat aggregate stats.
3. **Graceful shutdown + auto-reconnect:** Simulasi shutdown server. Client harus otomatis reconnect ke server lain dalam < 2 detik. Tampilkan counter reconnect di client.
4. **Health check endpoint:** Bikin endpoint /health yang return status server: uptime, connected clients, memory usage, Redis connection status. Pakai `process.memoryUsage()` dan ping Redis.